

1. Range GCD Query with Point Updates :

```
import java.util.*;

public class Main {
    static int[] arr;
    static int[] tree;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        arr = new int[n];
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        tree = new int[4 * n];
        build(0, 0, n - 1);

        int q = sc.nextInt();

        while(q-- > 0) {
            int a = sc.nextInt();

            if(a == 1) {
                int idx = sc.nextInt();
                int val = sc.nextInt();

                arr[idx] = val;
                update(0, 0, n - 1, idx, val);
            }
            else {
                int l = sc.nextInt();
                int r = sc.nextInt();

                System.out.println(query(0, 0, n - 1, l, r));
            }
        }
    }

    static void build(int node, int start, int end) {
        if(start == end) {
            tree[node] = arr[start];
            return;
        }
    }
```

```

    int mid = (start + end) / 2;
    build(2*node + 1, start, mid);
    build(2*node + 2, mid + 1, end);

    tree[node] = gcd(tree[2*node + 1], tree[2*node + 2]);
}
static void update(int node, int start, int end, int idx, int val) {
    if(start == end) {
        tree[node] = val;
        return;
    }
    int mid = (start + end) / 2;

    if(idx <= mid) {
        update(2*node + 1, start, mid, idx, val);
    }
    else {
        update(2*node + 2, mid + 1, end, idx, val);
    }

    tree[node] = gcd(tree[2*node + 1], tree[2*node + 2]);
}
static int query(int node, int start, int end, int l, int r) {
    if(r < start || l > end) {
        return 0;
    }
    if(l <= start && r <= end) {
        return tree[node];
    }
    int mid = (start + end) / 2;

    int left = query(2*node + 1, start, mid, l, r);
    int right = query(2*node + 2, mid + 1, end, l, r);

    return gcd(left, right);
}
static int gcd(int a, int b) {
    while(b != 0) {
        int temp = a % b;
        a = b;
        b = temp;
    }
    return a;
}
}

```

2. Range Product Query with Point Updates

```
import java.util.*;

public class Main {

    static long[] arr;
    static long[] tree;
    static long MOD = 1000000007L;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        arr = new long[n];
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextLong();
        }
        tree = new long[4 * n];
        build(0, 0, n - 1);

        int q = sc.nextInt();
        while(q-- > 0) {
            int type = sc.nextInt();
            if(type == 1) {
                int idx = sc.nextInt();
                long val = sc.nextLong();
                arr[idx] = val;
                update(0, 0, n - 1, idx, val);
            }
            else {
                int l = sc.nextInt();
                int r = sc.nextInt();
                System.out.println(query(0, 0, n - 1, l, r));
            }
        }
    }

    static void build(int node, int start, int end) {
        if(start == end) {
            tree[node] = arr[start] % MOD;
            return;
        }
        int mid = (start + end) / 2;
        build(2 * node + 1, start, mid);
        build(2 * node + 2, mid + 1, end);
        tree[node] = (tree[2 * node + 1] * tree[2 * node + 2]) % MOD;
    }
```

```

static void update(int node, int start, int end,int idx, long val) {

    if(start == end) {
        tree[node] = val % MOD;
        return;
    }
    int mid = (start + end) / 2;
    if(idx <= mid) {
        update( 2 * node + 1,start,mid, idx,val);
    }
    else {
        update( 2 * node + 2,mid + 1,end, idx,val);
    }

    tree[node] = (tree[2 * node + 1] * tree[2 * node + 2]) % MOD;
}

static long query(int node, int start, int end, int l, int r) {
    if(r < start || end < l) {
        return 1;
    }
    if(l <= start && end <= r) {
        return tree[node];
    }
    int mid = (start + end) / 2;
    long left = query(2 * node + 1, start, mid, l,r);
    long right = query(2 * node + 2, mid + 1, end, l, r);
    return (left * right) % MOD;
}
}

```

3. Range Bitwise OR Query with Point Updates

```

import java.util.*;

public class Main {
    static int[] arr;
    static int[] tree;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        arr = new int[n];
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        tree = new int[4 * n];
    }
}

```

```

build(0, 0, n - 1);

int q = sc.nextInt();
while(q-- > 0) {
    int type = sc.nextInt();
    if(type == 1) {
        int idx = sc.nextInt();
        int val = sc.nextInt();
        arr[idx] = val;
        update(0, 0, n - 1, idx, val);
    }
    else {
        int l = sc.nextInt();
        int r = sc.nextInt();
        System.out.println(query(0, 0, n - 1, l, r));
    }
}
}

static void build(int node, int start, int end) {
    if(start == end) {
        tree[node] = arr[start];
        return;
    }
    int mid = (start + end) / 2;
    build(2 * node + 1, start, mid);
    build(2 * node + 2, mid + 1, end);

    tree[node] = tree[2 * node + 1] | tree[2 * node + 2];
}

static void update(int node, int start, int end, int idx, int val) {
    if(start == end) {
        tree[node] = val;
        return;
    }
    int mid = (start + end) / 2;
    if(idx <= mid) {

        update(2 * node + 1, start, mid, idx, val);
    }
    else {
        update(2 * node + 2, mid + 1, end, idx, val);
    }

    tree[node] = tree[2 * node + 1] | tree[2 * node + 2];
}

static int query(int node, int start, int end, int l, int r) {

```

```

    if(r < start || end < l) {
        return 0;
    }
    if(l <= start && end <= r) {
        return tree[node];
    }
    int mid = (start + end) / 2;
    int left = query( 2 * node + 1, start, mid, l, r);

    int right = query( 2 * node + 2, mid + 1, end, l, r);

    return left | right;
}
}

```

5. First Element Greater Than or Equal to X in a Range

```

import java.util.*;

public class Main {
    static int[] arr;
    static int[] tree;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        arr = new int[n];
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        tree = new int[4 * n];
        build(0, 0, n - 1);

        int q = sc.nextInt();
        while(q-- > 0) {
            int type = sc.nextInt();
            if(type == 1) {
                int idx = sc.nextInt();
                int val = sc.nextInt();
                arr[idx] = val;
                update(0, 0, n - 1, idx, val);
            }
            else {
                int l = sc.nextInt();
                int r = sc.nextInt();
                int x = sc.nextInt();
            }
        }
    }
}

```

```

        System.out.println(
            query(0, 0, n - 1, l, r, x)
        );
    }
}

static void build(int node, int start, int end) {
    if(start == end) {
        tree[node] = arr[start];
        return;
    }
    int mid = (start + end) / 2;
    build(2 * node + 1, start, mid);
    build(2 * node + 2, mid + 1, end);
    tree[node] = Math.max(tree[2 * node + 1], tree[2 * node + 2]);
}

static void update(int node, int start, int end, int idx, int val) {
    if(start == end) {
        tree[node] = val;
        return;
    }
    int mid = (start + end) / 2;

    if(idx <= mid) {
        update(2 * node + 1, start, mid, idx, val);
    }
    else {
        update(2 * node + 2, mid + 1, end, idx, val);
    }

    tree[node] = Math.max(tree[2 * node + 1], tree[2 * node + 2]);
}

static int query(int node, int start, int end, int l, int r, int x) {
    if(r < start || end < l) {
        return -1;
    }
    if(tree[node] < x) {
        return -1;
    }
    if(start == end) {
        return start;
    }
    int mid = (start + end) / 2;
    int left = query(2 * node + 1, start, mid, l, r);

    if(left != -1) {

```

```

        return left;
    }

    return query(2 * node + 2, mid + 1, end, l, r);
}
}

```

6. Count Zeros in a Range with Point Updates

```

import java.util.*;

public class Main {
    static int[] arr;
    static int[] tree;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        arr = new int[n];
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        tree = new int[4 * n];
        build(0, 0, n - 1);

        int q = sc.nextInt();
        while(q-- > 0) {
            int type = sc.nextInt();
            if(type == 1) {
                int idx = sc.nextInt();
                int val = sc.nextInt();
                arr[idx] = val;
                update(0, 0, n - 1, idx, val);
            }
            else {
                int l = sc.nextInt();
                int r = sc.nextInt();
                System.out.println(query(0, 0, n - 1, l, r));
            }
        }
    }

    static void build(int node, int start, int end) {
        if(start == end) {
            if(arr[start] == 0) {
                tree[node] = 1;
            }
            else {

```



```

        tree[node] = 0;
    }
    return;
}
int mid = (start + end) / 2;

build(2 * node + 1, start, mid);
build(2 * node + 2, mid + 1, end);

tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
}

static void update(int node, int start, int end, int idx, int val) {
    if(start == end) {
        if(val == 0) {
            tree[node] = 1;
        }
        else {
            tree[node] = 0;
        }
        return;
    }
    int mid = (start + end) / 2;

    if(idx <= mid) {
        update(2 * node + 1, start, mid, idx, val);
    }
    else {
        update(2 * node + 2, mid + 1, end, idx, val);
    }

    tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
}

static int query(int node, int start, int end, int l, int r) {
    if(r < start || end < l) {
        return 0;
    }
    if(l <= start && end <= r) {
        return tree[node];
    }
    int mid = (start + end) / 2;

    int left = query(2 * node + 1, start, mid, l, r);
    int right = query(2 * node + 2, mid + 1, end, l, r);

    return left + right;
}

```

